Presentation Script

Word-for-word speaking notes for all 16 slides — NexCare_Capstone_Presentation.pptx (v2, code-deep-dive edition)

Slide-by-Slide Script | Code Walkthroughs | Timing Guide (~16 min) | Anticipated Questions

How to use this document

The deck was redesigned to be more technical — it now includes two dedicated "Code Deep Dive" slides showing real backend and frontend code, a proper database/API reference slide, and a testing summary slide, instead of plain bullet lists. Each slide below has: **What to Say**, **Key Points to Hit**, and **If Asked** (follow-up Q&A). For the two code slides, read the code out loud line-by-line the first time you practice — once you can explain every line without looking at notes, you're ready. Total run-time target: **14–17 minutes**, leaving time for Q&A.

1 Cover — NexCare

~40 sec

WHAT TO SAY

"Good morning/afternoon everyone. My name is Rishitha Manyam, and today I'll be presenting my capstone project for the Wipro RGA Program 2025 — it's called NexCare, an Interactive Dashboard System for Customer Service Operations. It's built entirely with the Java Full Stack technologies from the program — Angular 17 on the frontend, Spring Boot 3.2.5 on the backend, secured end-to-end with JWT."

KEY POINTS TO HIT

- Name + program name clearly, project name once.
- Naming the exact versions (Angular 17, Spring Boot 3.2.5, JWT) up front signals technical precision.

2 Agenda

~30 sec

WHAT TO SAY

"Here's the roadmap — I'll cover the problem and my solution, the tech stack and architecture, then go deep on security with an actual JWT flow diagram. After that I'll walk through real backend and frontend code, the database design and API endpoints, the four role-based dashboards, key features, the challenges I hit while building this, a summary of my testing, and close with key learnings and conclusions."

KEY POINTS TO HIT

- Specifically mention "I'll walk through real code" — this previews the two code-deep-dive slides and sets evaluator expectations that this isn't just a slideshow of bullet points.

3 Problem Statement

~55 sec

WHAT TO SAY

"Telecom customer service teams in India today rely on disconnected tools. Representatives track tickets in spreadsheets, so there's no unified, searchable view of what's happening with any customer. Managers have no real-time dashboard showing response time, resolution time, or workload per representative. Customers have no visibility into their own ticket status, so they call back repeatedly just to ask for an update — which eats into representative time that should go to new issues. There's also no role-based access control in ad-hoc tooling — any staff member could potentially see data belonging to any customer, which is both a privacy and a compliance risk. And when there's a service outage, it's communicated informally and untargeted — every customer gets notified, even ones in completely unaffected cities. My goal was to build one secure, role-based, real-time application that solves all five of these problems together."

KEY POINTS TO HIT

- Five distinct problems, each later mapped to a specific NexCare feature — say them in order so the "Solution" slide feels like a direct answer.

IF ASKED

"Is this based on a real company?" → "It's modeled on common pain points reported across small and mid-sized Indian telecom service desks — a representative problem, not one specific company's internal system."

4 Solution Overview — NexCare

~65 sec

WHAT TO SAY

"NexCare is a full-stack Java and Angular web application built specifically for Indian telecom customer service operations. It gives four role-based dashboards — Admin, Manager, Representative, and Customer. The entire API is JWT-secured, so every request is authenticated and every user only ever sees data they're authorised for. Ticket management happens with a full audit trail — every status change is timestamped automatically. There's a live chatbot covering twelve common customer intents, reducing repetitive load on representatives. I built live analytics using Chart.js — bar and doughnut charts — for performance monitoring. Outage alerts are smart: they only surface to customers in the affected city. The whole application is localised for India — INR currency, Indian city names, Indian phone number formats. And the database requires zero setup — it's an H2 file database that auto-seeds all demo data the first time the application runs."

KEY POINTS TO HIT

- Read the eight feature cards left-to-right, top-to-bottom — they map directly back to the five problems from slide 3.
- End on "zero-setup database" — a strong, concrete, memorable closing fact for this slide.

5 Tech Stack

~75 sec

WHAT TO SAY

"On the frontend: Angular 17 using standalone components, meaning there's no NgModule anywhere in this codebase — that's the modern Angular approach. Everything is strict TypeScript. Chart.js powers the bar and doughnut charts. Styling is plain CSS3 with a lilac theme and cubic-bezier animation curves. Routing uses Angular Router with lazy loading and hash-based URLs. RxJS handles reactive state and communication between components that aren't directly related. And a proxy configuration makes the dev server forward any '/api' call straight to port 8080, avoiding CORS issues during development. On the backend: Spring Boot 3.2.5 as the REST API, running on an embedded Tomcat server. Spring Security enforces a JWT filter chain on every request. I used the JJWT library, version 0.12.3, for generating, signing, and validating tokens. Spring Data JPA is the ORM for every entity — no hand-written SQL anywhere. The database is H2 running in file mode, so

no external database server is needed at all. Passwords are hashed with BCryptPasswordEncoder — never stored in plain text. And the whole project builds with Maven."

KEY POINTS TO HIT

- Say "standalone components, no NgModule" explicitly — signals modern Angular knowledge.
- Say "embedded Tomcat" for Spring Boot — shows you know it's not a separately deployed server.

IF ASKED

"Why Angular over React?" → "Angular is a complete framework — routing, forms, and an HTTP client are all built in. React is just a library, so you'd need several third-party packages for the same coverage. For an enterprise dashboard like this, Angular's structure made more sense."

"Why H2 and not MySQL/PostgreSQL?" → "H2 needed zero installation — it runs as a file inside the project, making the app instantly runnable for demo and evaluation. For real production use I'd migrate to PostgreSQL, which I list under future scope."

6 System Architecture

~60 sec

WHAT TO SAY

"NexCare follows a three-tier architecture, shown here top to bottom. The Presentation layer is the Angular 17 single-page application — components, the router, functional guards, and functional interceptors. Below that is the Service/API layer — Spring Boot REST controllers: AuthController, TicketController, AnalyticsController, OutageController. At the bottom is the Data layer — Spring Data JPA on top of an H2 file database, storing the User, Ticket, Outage, and Notification entities. These layers communicate purely over HTTP using JSON — a proxy handles it in development, direct CORS configuration handles it in production. And critically, every single request between frontend and backend carries a JWT bearer token, validated on every protected endpoint, not just at login."

KEY POINTS TO HIT

- Point at each tier on screen as you describe it — this slide is a literal top-to-bottom diagram, follow it visually.
- Close on the orange callout box: "JWT bearer token on every request" — this is the bridge into the next slide.

IF ASKED

"Why three tiers instead of one app?" → "Separating frontend and backend means each can be deployed, scaled, or even rewritten independently — and the same backend API could serve a mobile app later with zero changes."

7 Security Implementation — JWT Authentication Flow

~95 sec (most-asked-about slide)

WHAT TO SAY

"This is the most important technical slide, so I'll walk through all eight steps. One — the user submits email and password through the Angular Sign-In form. Two — my AuthController calls a UserDetailsService, which uses BCrypt to verify the submitted password against the hashed password in the database; the plain password is never stored anywhere. Three — JwtUtil generates a signed token containing the user's ID, email, role, and a 24-hour expiry. Four — that token is returned to the frontend and stored in sessionStorage, which clears automatically when the browser tab closes

— a deliberate choice over localStorage. Five — from that point on, a JWT Interceptor automatically attaches the token as an 'Authorization: Bearer' header to every outgoing request, so I never manually add it anywhere. Six — on the backend, JwtAuthFilter validates that token's signature on every protected endpoint before the request even reaches a controller. Seven — on the frontend, an Auth Guard checks the role from that token before the Angular router allows navigation to a protected page. And eight — logging out clears sessionStorage, so even an intercepted token becomes unusable immediately."

KEY POINTS TO HIT

- Walk all 8 numbered cards in order — evaluators will probe this slide hardest.
- Emphasize: passwords never stored in plaintext (BCrypt one-way hash).
- Emphasize: enforcement happens at BOTH the frontend guard AND backend filter — two independent layers, not one.

IF ASKED

"What's inside the JWT exactly?" → "Three Base64-encoded parts joined by dots: a header naming the signing algorithm, a payload with user ID/email/role/expiry, and a signature. The signature is what prevents tampering — regenerating a valid one requires a secret key only the server holds."

"What happens after the token expires?" → "Any further API call returns 401, the frontend interceptor catches that and redirects the user back to login to re-authenticate."

8 Code Deep Dive — Backend (JWT Filter & Ticket Service)

~90 sec

WHAT TO SAY

"Let me show actual code instead of just describing it. On the left is JwtAuthFilter — notice it extends OncePerRequestFilter, which is a Spring Security base class that guarantees this filter runs exactly once per incoming request, before that request reaches any controller. Inside doFilterInternal, I extract the token from the request, and if it's present and valid, I pull the role out of it and set the Spring Security authentication context — then the request continues down the filter chain. If the token is missing or invalid, I simply don't set the authentication, so the request gets rejected further down by the role check. On the right is TicketService.updateStatus — this is where the response-time metric I mentioned earlier actually gets calculated. When a ticket moves to IN_PROGRESS for the first time, I capture the current timestamp, then calculate the duration in hours between ticket creation and this moment using Java's Duration class, and store that on the ticket. This number is never typed in by a human — it's purely derived from two timestamps the system already has."

KEY POINTS TO HIT

- Name the exact class (OncePerRequestFilter) — shows you understand Spring Security internals, not just "there's a filter."
- Walk the ticket service logic conditionally: "if it's the FIRST time moving to IN_PROGRESS" — explain why the null check on firstResponseAt matters (prevents recalculating on every subsequent status change).
- Close with the callout box: explicit getters/setters instead of a code-generation library, for build portability across JDK versions.

IF ASKED

"What if the token is expired but still has a valid signature?" → "JwtUtil.isValid() checks both the signature AND the expiry claim — an expired token fails validation even with a perfectly correct signature, so isValid() returns false and the filter never sets authentication."

"Why Duration instead of just subtracting LocalDateTime fields?" → "Duration.between() handles date-boundary and time-zone edge cases correctly — manual subtraction of LocalDateTime fields is error-prone across day boundaries."

9 Code Deep Dive — Frontend (Guard & Interceptor)

~85 sec

WHAT TO SAY

"On the left is authGuard, written as a plain function — this is the Angular 17 functional style, using inject() instead of a class with constructor injection. It pulls the current user's role and checks two things: is the user logged in at all, and if the route declares an allowed-roles list, is this user's role in that list? If either check fails, it redirects to login and returns false, blocking navigation. On the right is jwtInterceptor, also a plain function. Every single outgoing HTTP request passes through this — it reads the token from sessionStorage and, if present, clones the request with an Authorization header attached. This is registered once in app.config.ts and applies globally, so no individual service anywhere in the app needs to remember to attach a token manually."

KEY POINTS TO HIT

- Emphasize "functional pattern" and inject() by name — this is the detail that distinguishes Angular 17 knowledge from older Angular tutorials.
- Make the distinction explicit: the guard is a UX convenience, the backend filter from the previous slide is the actual security boundary — repeating this ties slides 7, 8, and 9 together into one coherent security story.

IF ASKED

"What stops someone from bypassing the guard by calling the API directly with curl or Postman?" → "Nothing on the frontend — and that's by design. The guard never touches the server. The backend's JwtAuthFilter and @PreAuthorize checks are what actually enforce security; the frontend guard only improves the user experience by not letting the UI render a page it doesn't need to."

10 Database Design & API Reference

~70 sec

WHAT TO SAY

"Four tables back this whole application. Users — all four roles in one table distinguished by a role enum, with passwords stored as BCrypt hashes. Tickets — storing customer ID, rep ID, manager ID, status, the three audit timestamps, and the two computed metrics I just walked through in the code. Outages — location, affected areas, domain, and severity. Notifications — recipient user ID, message, and a read flag, created automatically by the ticket and outage services as side effects, not by any direct user action. For the API, here are the core endpoints. POST /api/auth/login returns a JWT. POST /api/auth/register creates a new customer. GET /api/tickets returns tickets already filtered by the caller's role on the server — a customer only ever gets their own tickets back, never a flag the frontend has to enforce. PUT /api/tickets/{id} updates status and stamps the audit fields automatically. GET /api/analytics/{role} returns a pre-aggregated statistics map for dashboard charts. And GET /api/outages/location filters outages down to one city for the customer-facing alert banner."

KEY POINTS TO HIT

- Emphasize "filtered by role on the server" for GET /api/tickets — this is the data-isolation guarantee from the problem statement, made concrete.
- Color-coded HTTP method pills (POST green, GET blue, PUT orange) — just read method + path + one-line description per row, don't over-explain each one.

IF ASKED

"Why one Users table for all four roles?" → "All roles share the same core fields — name, email, password, phone, location — so one table with a role enum column avoids duplicate schema and keeps authentication a single, simple query regardless of who's logging in."

11 Role-Based Dashboards

~55 sec

WHAT TO SAY

"Four roles, four dedicated dashboards — not one generic dashboard with hidden sections. Admin manages all employees, views every ticket system-wide, reports outages, and sees the full analytics overview. Manager monitors their team's tickets, resolves outages, views individual rep performance metrics, and manages escalations. Representative views and updates only their assigned tickets,

searches customers by name or phone, and tracks their own resolution time — plus they're forced to change their password on first login. Customer raises tickets, tracks status live, rates closed tickets, and uses the chatbot and outage alerts."

KEY POINTS TO HIT

- Say "four dedicated dashboards, not one generic dashboard" — this is a deliberate design decision worth stating explicitly.

IF ASKED

"What stops a Representative from changing the URL to reach the Admin page?" → "The Angular route guard from slide 9 redirects them immediately based on role. And even if they bypassed the frontend entirely and called the API directly, the backend's @PreAuthorize annotations reject the request with a 403 — the frontend guard is convenience, the backend is enforcement."

12 Key Features Walkthrough

~80 sec (pairs well with a live demo)

WHAT TO SAY

"Ten features worth highlighting. The login page has a full-page animated lilac overlay transition between Sign In and Sign Up, built with CSS cubic-bezier easing. Every dashboard route is lazy-loaded using Angular's loadComponent — code only downloads when a user actually visits that route. Tab navigation uses URL query parameters, so every tab is bookmarkable and survives a refresh. Chart.js powers live bar charts for tickets-by-domain and doughnut charts for status breakdown. The outage alert banner only appears to customers whose city matches the outage location. The AI chatbot recognises twelve intent keywords and can be triggered from the sidebar via an RxJS Subject event bus. Customers rate closed tickets, and that score immediately appears on the representative's own dashboard. Representatives search customers with a 400-millisecond debounce so we're not spamming the API on every keystroke. First-time representative logins are detected and they're prompted to change their password. And the whole thing is India-localised — Rupee symbols, Indian cities, Indian phone formats."

KEY POINTS TO HIT

- If time allows, pause here and actually demo 2-3 of these live — debounced search and the outage banner are the most visually convincing.
- Debounce and lazy-loading show performance awareness, not just feature-completeness.

IF ASKED

"Is the chatbot a real AI model?" → "No — it's keyword/intent matching covering 12 common questions. It's instant and self-contained, with no external API dependency. Upgrading it to an NLP-based model is in my future scope."

13 Challenges & Solutions

~75 sec

WHAT TO SAY

"Five real challenges. First — the sidebar and chatbot are sibling components with no parent-child relationship, so direct DOM manipulation kept getting overridden by Angular's own change detection. I solved it with a shared ChatService using an RxJS Subject as an event bus. Second — I originally used scrollIntoView for tab navigation, which broke on page refresh; I switched entirely to Angular

Router query parameters. Third — Chart.js needs an actual canvas element to exist before it can draw, but my charts were inside Angular @if blocks that hadn't rendered yet; I fixed this with a short setTimeout after the tab switch so the DOM element exists first. Fourth — Angular's dev server on port 4200 and Spring Boot on 8080 triggered CORS errors; I added a proxy.conf.json so the dev server transparently forwards every /api request. And fifth — my database seed was re-running on every restart; I fixed that with an existsById check in DataInitializer so it only seeds when the admin account doesn't already exist."

KEY POINTS TO HIT

- This slide proves you debugged real problems rather than following a tutorial — give each pair a crisp "why it broke" then "what fixed it," don't rush past it.

IF ASKED

"What was the hardest bug to fix?" → "The Chart.js timing issue — it wasn't an error message, the chart would just silently not appear, so it took a while to realise the canvas element didn't exist yet when the chart code ran."

14 Testing Summary

~50 sec

WHAT TO SAY

"Before treating this as demo-ready, I manually tested seven core scenarios. Valid admin login issues a JWT and redirects correctly. An incorrect password returns 401. A customer trying to access /admin directly through the URL gets redirected by the route guard. When a representative resolves a ticket, resolutionTimeHrs is calculated automatically. Reporting an outage in Mumbai means only Mumbai customers see the alert — Delhi and Bangalore customers see nothing. Token expiry after 24 hours correctly forces a re-login. And restarting the application twice in a row does not duplicate the seed data, confirming DataInitializer's existsById guard works as intended. All seven passed."

KEY POINTS TO HIT

- Say "manually tested" honestly — don't claim automated test suites exist if they don't; the formal report lists this as future scope, which is the correct, honest answer if asked.

IF ASKED

"Do you have automated unit tests?" → "Not yet — testing was done manually against these seven scenarios for this capstone phase. Adding JUnit tests for the backend and Karma/Jasmine tests for the frontend is explicitly listed in my future scope."

15 Key Learnings & Conclusion

~65 sec

WHAT TO SAY

"This project taught me things coursework alone couldn't. How Spring Security combined with JWT enables stateless authentication that scales without server-side session storage. Angular's standalone component model, which removes all the NgModule boilerplate older tutorials are full of. That RxJS Subjects are the correct pattern for communication between unrelated components. Functional interceptors and guards as the modern Angular 17 style. How a proxy configuration cleanly bridges frontend and backend in development. That Chart.js has to respect Angular's own rendering lifecycle, not fight it. And relying on @PostConstruct for dependable startup behaviour. To conclude — NexCare is a production-grade Java Full Stack application: secure, role-based, real-time, purpose-built for Indian telecom operations. It follows a clean three-tier architecture, requires zero manual setup, runs with two commands after installing prerequisites, and covers the full software

development lifecycle — design, development, security, testing, and documentation. I've also prepared comprehensive supporting material: a setup guide, a technical Q&A reference, a full code walkthrough, and a formal capstone report."

KEY POINTS TO HIT

- End on "zero manual setup, two commands" — concrete, memorable closing fact.

16 Thank You / Q&A

~15 sec + open floor

WHAT TO SAY

"Thank you for your time. That concludes my presentation on NexCare. I'd be happy to take any questions, and I'm glad to walk through a live demo or any part of the code in more detail."

KEY POINTS TO HIT

- Smile, slow down, make eye contact. Only offer a live demo if you're genuinely ready to do it.

Final Pre-Presentation Checklist

- ☐ Practiced the full script out loud at least twice, end to end, with a timer
- ☐ Can explain every line of both Code Deep Dive slides (8 and 9) without reading from notes
- ☐ Backend running locally and confirmed reachable (login test with admin@csd.com works)
- ☐ Frontend running locally at http://localhost:4200 and confirmed loading
- ☐ Demo credentials memorised or written on a sticky note: admin@csd.com / admin123
- ☐ Laptop charger plugged in, brightness up, notifications silenced before presenting
- ☐ PPTX opened and tested in Presenter View at least once beforehand
- ☐ Read through the Technical Q&A Guide (docs/02) the night before for extra confidence
- ☐ Know which 2-3 features you'll demo live on slide 12 if time allows